CIS*3110 Operating Systems

Week 5

Lecture #8 Virtualization: Paging

Prof. Yulia Kotseruba

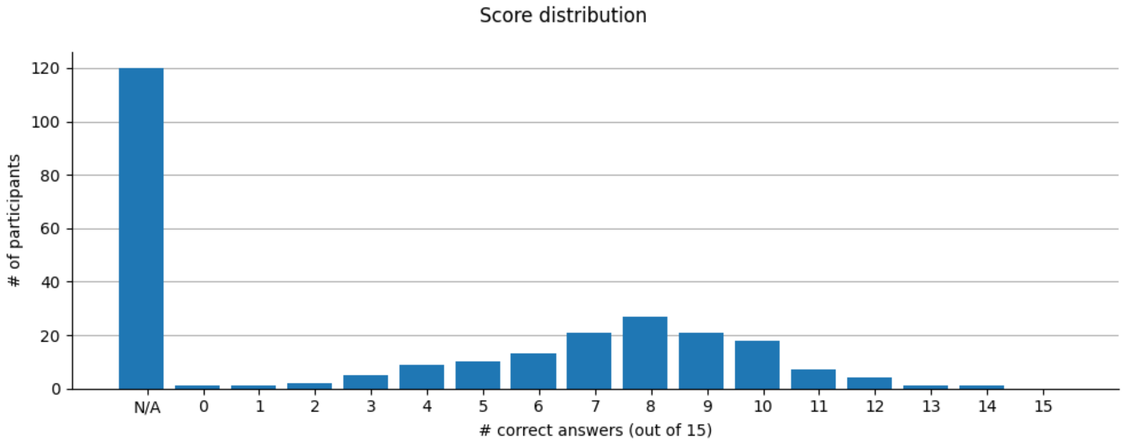
February 3, 2026

Quiz 6

- **50%** average (down from 61% in quiz 5)
- 3 hardest questions:
  - (9%) What is the output of this code? a) OK? OK! b) OK! c) runtime error d) segfault
  - (17%) Which process has a memory leak? a) child b) parent c) neither d) both
  - (28%) What does this code print? a) hi bye b) bye bye c) hi bye bye d) bye

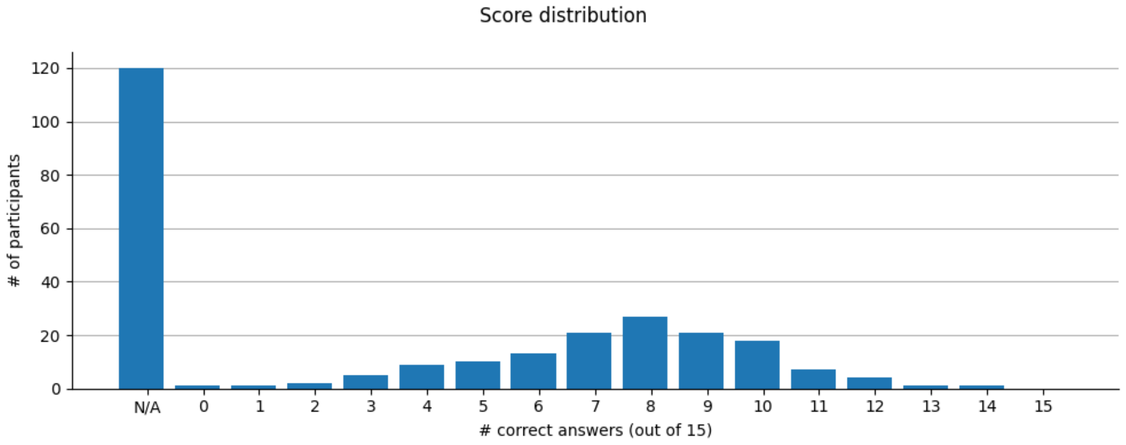
```c
int main(){
    if (fork()==0){
        free(-1);
        printf("OK?\n");
    } else {
        printf("OK!\n");
    }
    exit(0);
}
```

```c
int main(){
    void *p = malloc(10);
    if(fork()==0){
        execlp("sleep","sleep","1",NULL);
    }
    free(p);
    return 0;
}
```



Quiz 6

- **50%** average (down from 61% in quiz 5)
- 3 hardest questions:
  - (9%) What is the output of this code? a) OK? OK! b) OK! c) runtime error d) segfault
  - (17%) Which process has a memory leak? a) child b) parent c) neither d) both
  - (28%) What does this code print? a) hi bye b) bye bye c) hi bye bye d) bye




```
int main() {
  if (fork() == getpid()) {
    printf("hi \n");
  }
  printf("bye \n");
}
```

Recap

- **Memory virtualization**
  - Processes see their memory as contiguous and virtually unlimited
  - Processes are not aware of their memory location in physical memory
  - OS manages allocation and access to memory to protect processes from each other and itself from processes
- **Address space** - a chunk of memory allocated by the OS for each process
  - processes can allocate/deallocate memory within that chunk
- **Memory API** - `malloc()`, `free()` (which call `brk()`, `sbrk()`, and `mmap()`)

Recap

- How do we **translate** virtual addresses to physical memory addresses?
  - **dynamic** hardware-based **relocation** using base and bounds registers
- How do processes know which memory is available?
  - **free-list** - a linked list of free blocks
- How do we reduce **internal segmentation** (waste of free space between heap and stack in virtual address space)?
  - **segmentation** - 3 pairs of base and bounds registers for stack, heap, and text
  - segment address is **embedded** in the virtual address

Recap

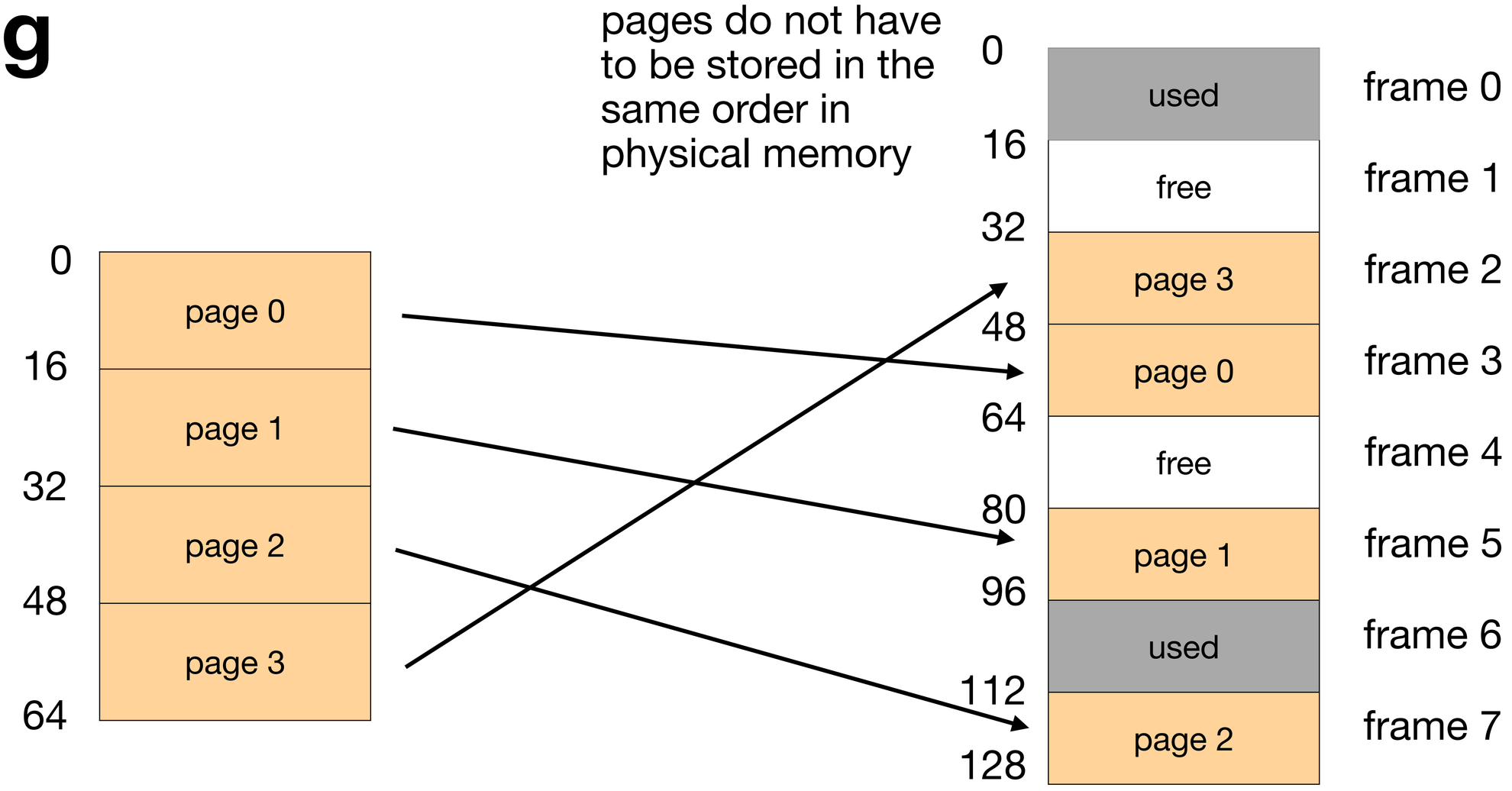
- How do we use memory most efficiently and avoid fragmentation?
- **Internal fragmentation** - free space within the address space of processes
  - avoid overallocating memory, difficult to control
- **External fragmentation** - free space between address spaces of processes
  - Option #1 - **compact** memory periodically
  - Option #2 - better **memory allocation** strategies

Paging

- Problem: **External fragmentation** is caused by **segmentation**, i.e. allocating/deallocating **variable** size blocks of memory
- Solution: **Paging** - divide memory into **equally** sized chunks
  - Break physical memory into **frames** of **fixed** size
  - Break virtual memory into **pages** of the **same** size

Paging

pages do not have to be stored in the same order in physical memory

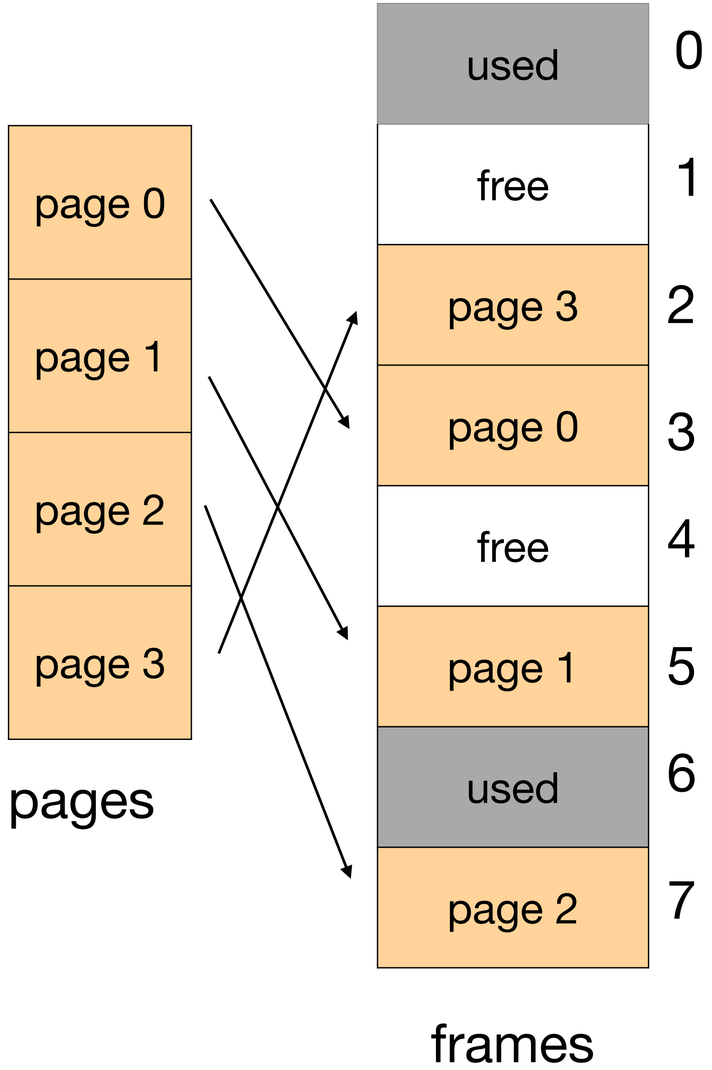


64-byte virtual address space of some process

128-byte physical memory

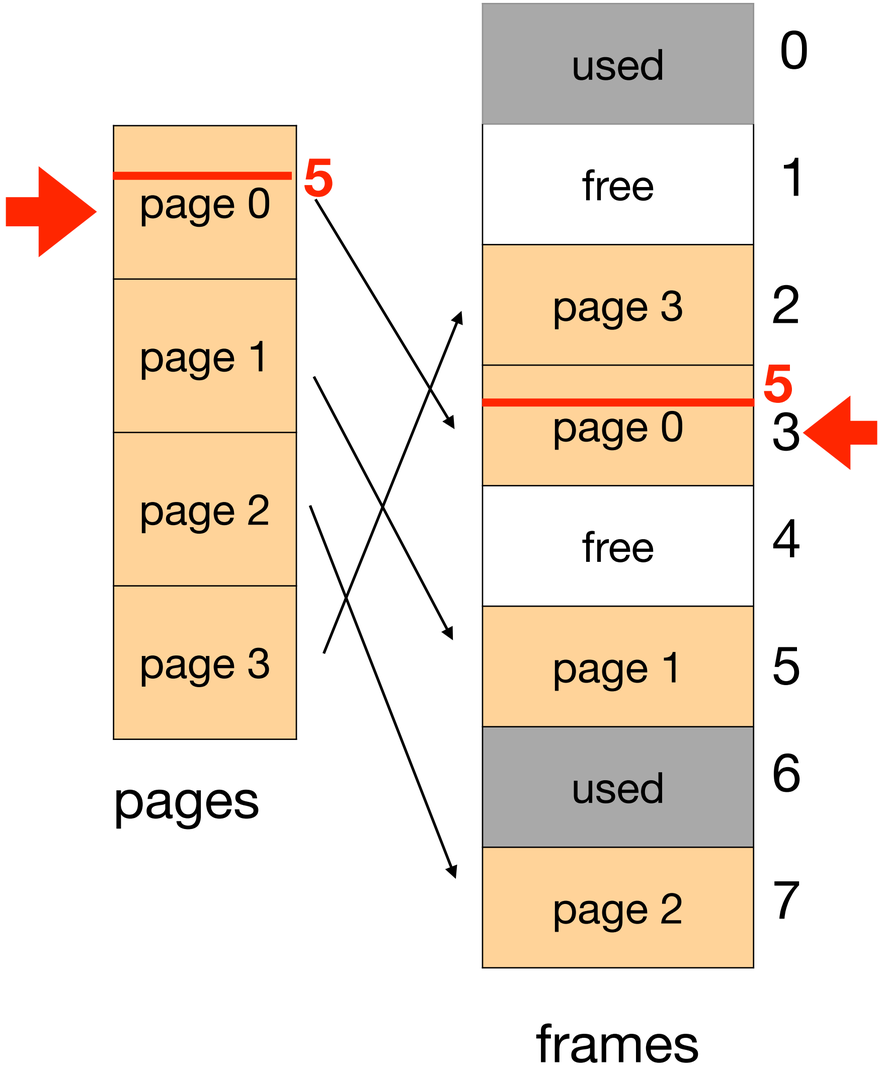
Paging

- Paging is segmentation with all chunks being equal
- Pros:
  - all blocks are the **same size**
  - easy to allocate - take **first empty** block
  - does **not** need base and bounds hardware registers
- Cons:
  - does **not** eliminate internal fragmentation
  - requires **a lot** of memory



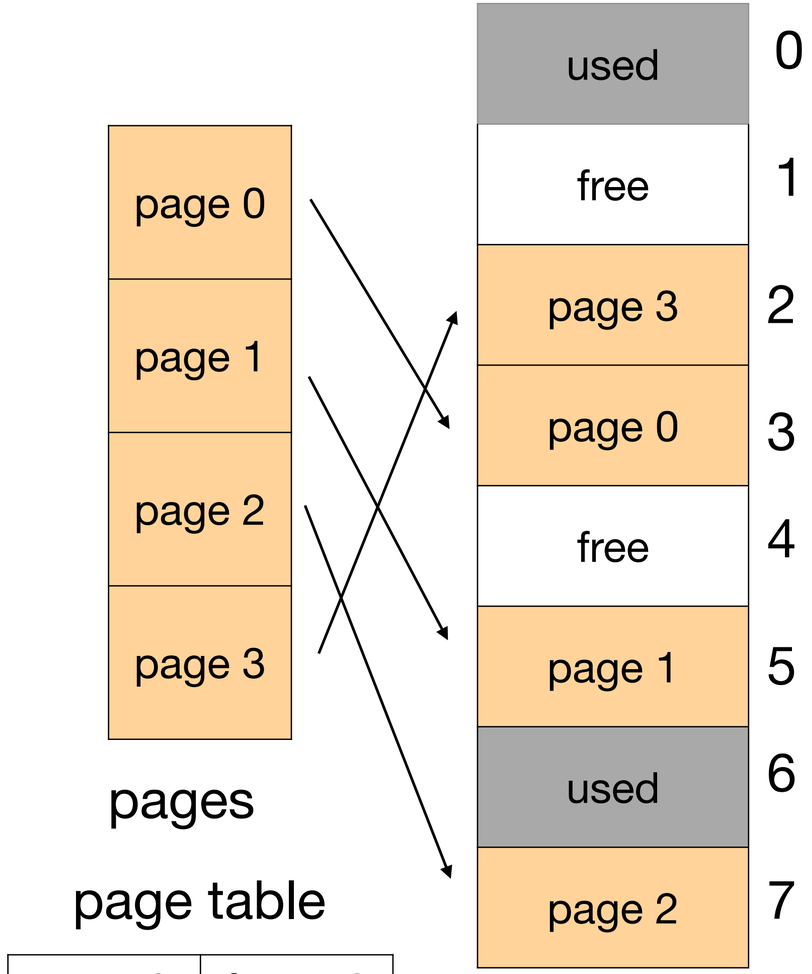
Paged addresses

- **Address translation**
  - find page number (e.g. page 0)
  - find a corresponding frame (e.g. 3)
  - find an offset within the page number (e.g. 5)
  - find the same offset within the frame number
- What is needed for this
  - **page table** - a **mapping** between pages and frames



Paged addresses

- **Page table**
  - a mapping between pages and frames
  - unique for **each** process
- Similar to segmentation, virtual address contains virtual page number (VPN) and offset
  - VPN can be looked up in the page number
  - The offset is the same
  - Bounds are known (page size is fixed)



page table

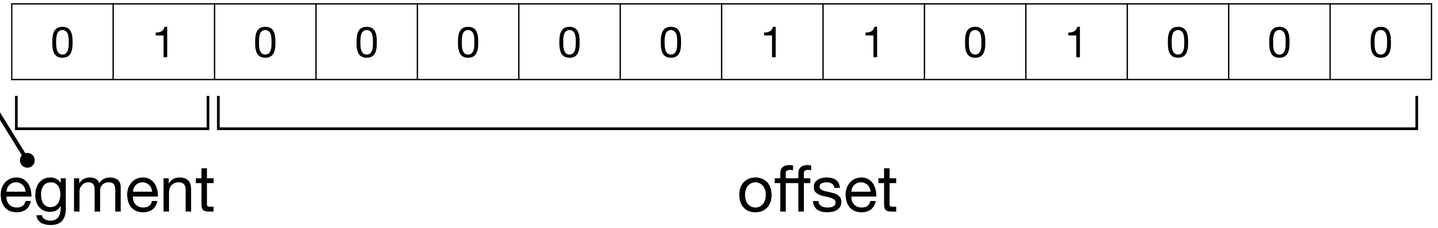
page 0	frame 3
page 1	frame 5
page 2	frame 7
page 3	frame 2

Paged addresses

recall segmentation

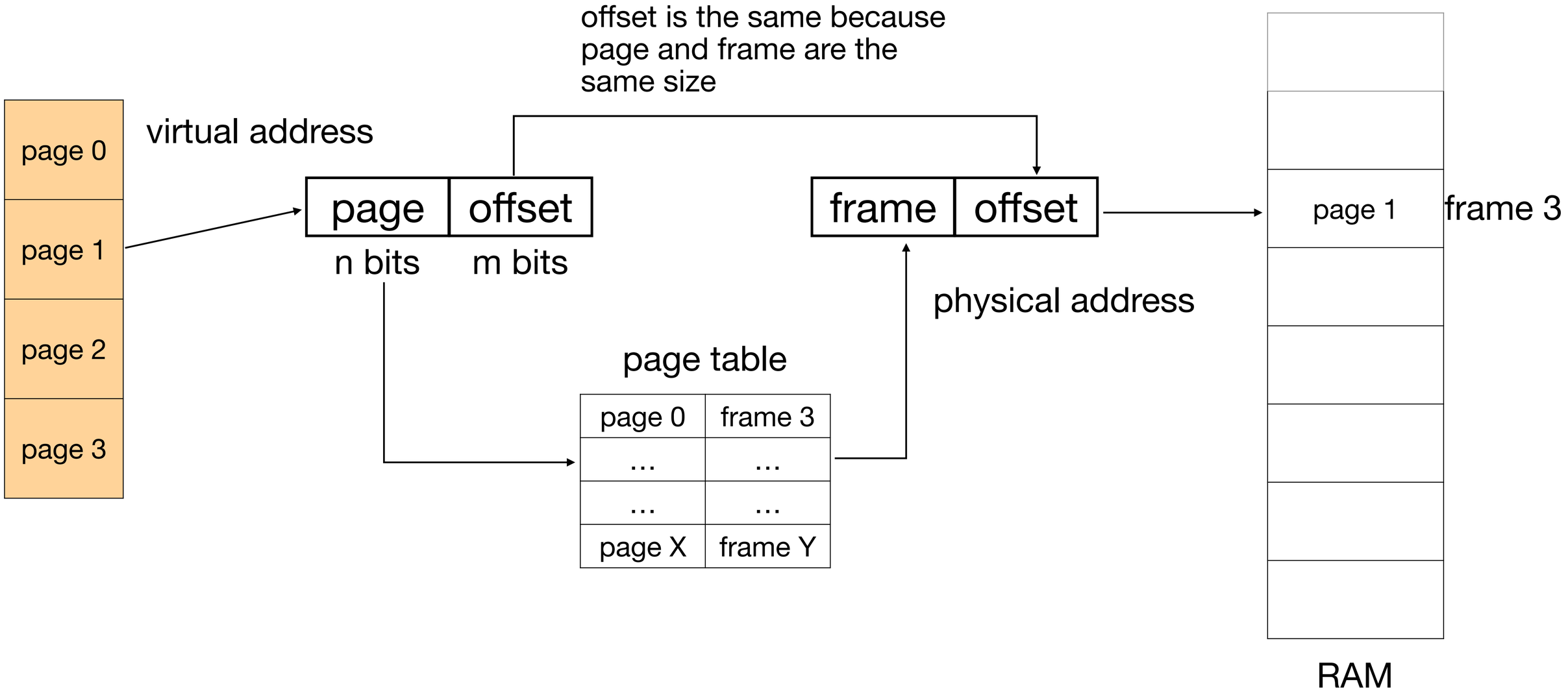
two top bits represent the segment (e.g. 0 0 for code, 0 1 for heap, 1 1 for stack)

virtual address



use base and bounds to check if offset is correct in physical memory

Paged addresses



Paged addresses

page	offset
n bits	m bits

- Problem: How many bits to use for encoding pages and offsets?
  - need enough bits to **address all pages**
  - need enough bits to access the **address space** of the process
  - depends on the **page size** and **total** amount of **memory**

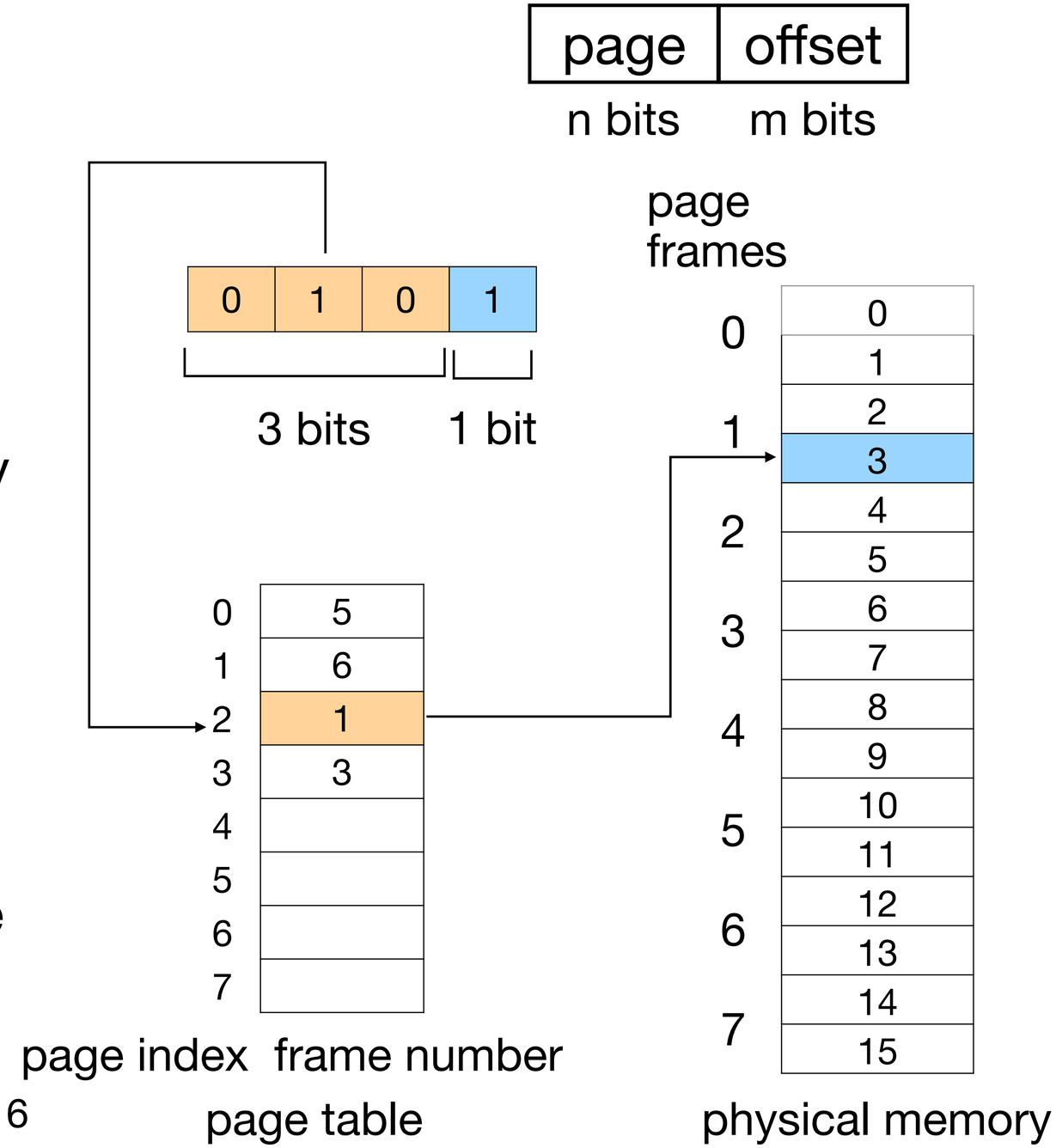
Paged addresses

page	offset
n bits	m bits

- 4-bit system - each address is 4 bits
- 2^4=16 different addresses
- 16 bytes of byte-addressable memory
- If we choose page size of 2 bytes
  - 16/2 = 8 pages
  - n = 3 - 3 bits to represent page
  - m = 1 - 1 bit for offset into the page

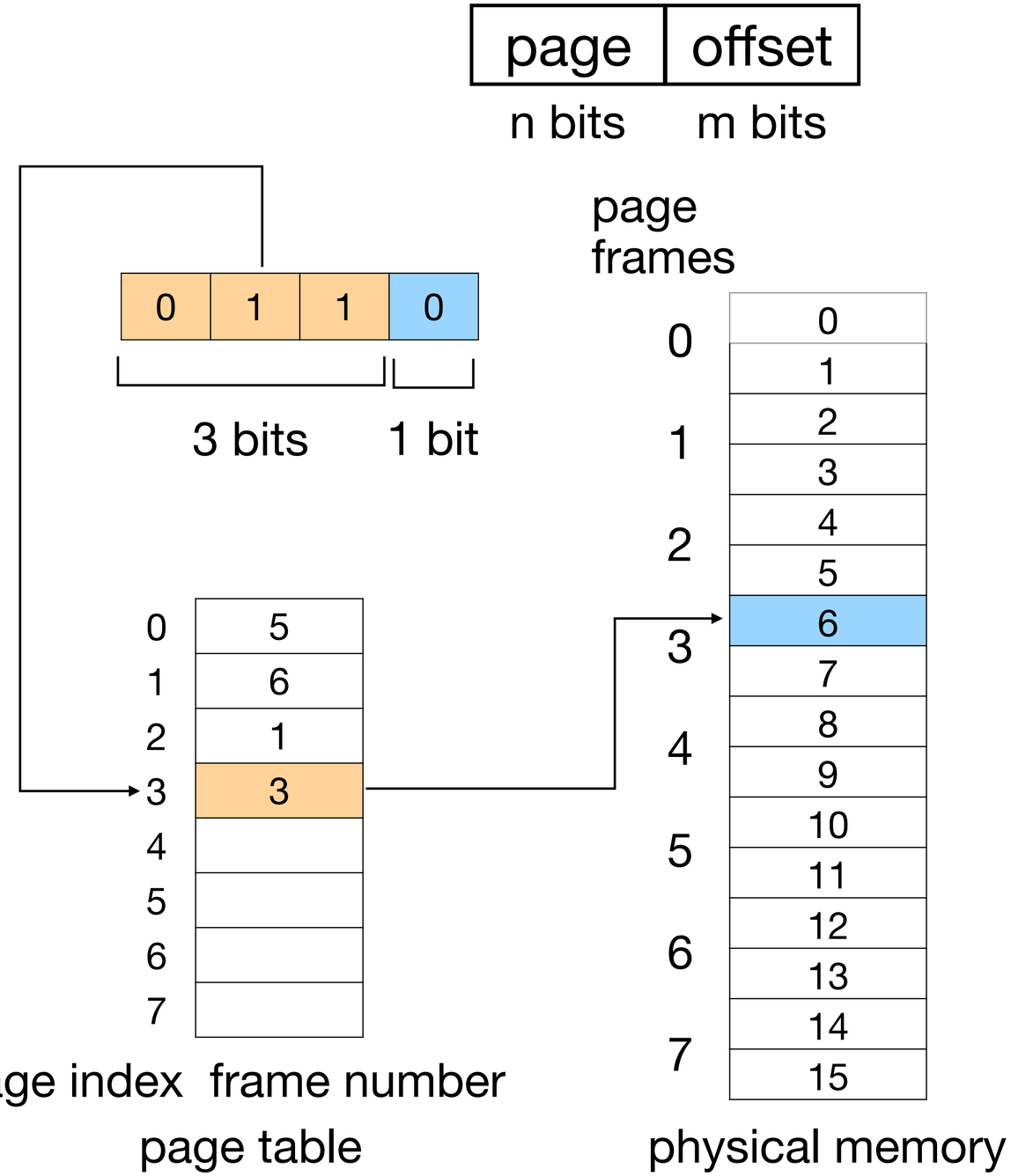
Paged addresses

- 4-bit system - each address is 4 bits
- 2^4=16 different addresses
- 16 bytes of byte-addressable memory
- If we choose page size of 2 bytes
  - 16/2 = 8 pages
  - n = 3 - 3 bits to represent page
  - m = 1 - 1 bit for offset into the page



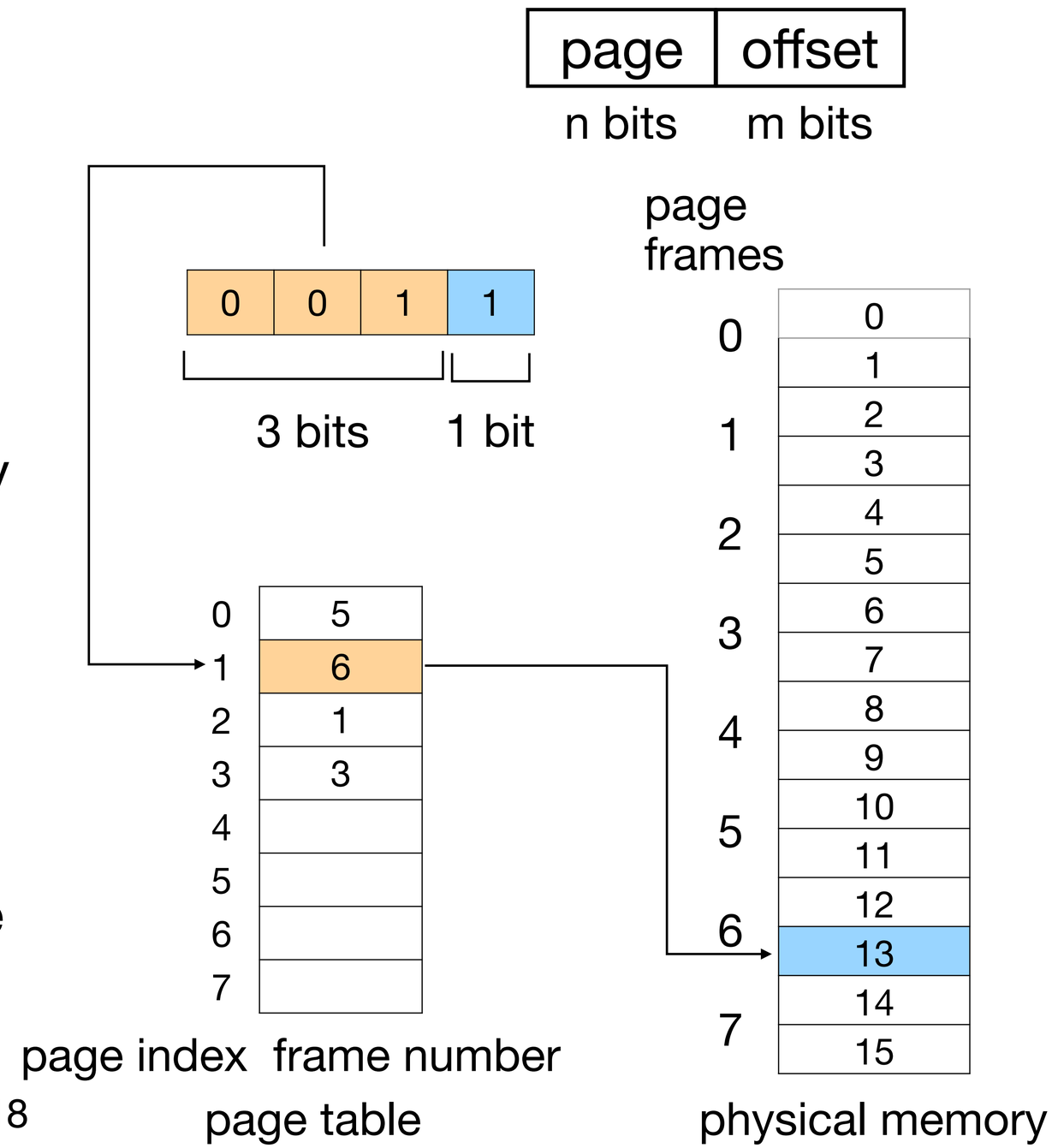
Paged addresses

- 4-bit system - each address is 4 bits
- 2^4=16 different addresses
- 16 bytes of byte-addressable memory
- If we choose page size of 2 bytes
  - 16/2 = 8 pages
  - n = 3 - 3 bits to represent page
  - m = 1 - 1 bit for offset into the page



Paged addresses

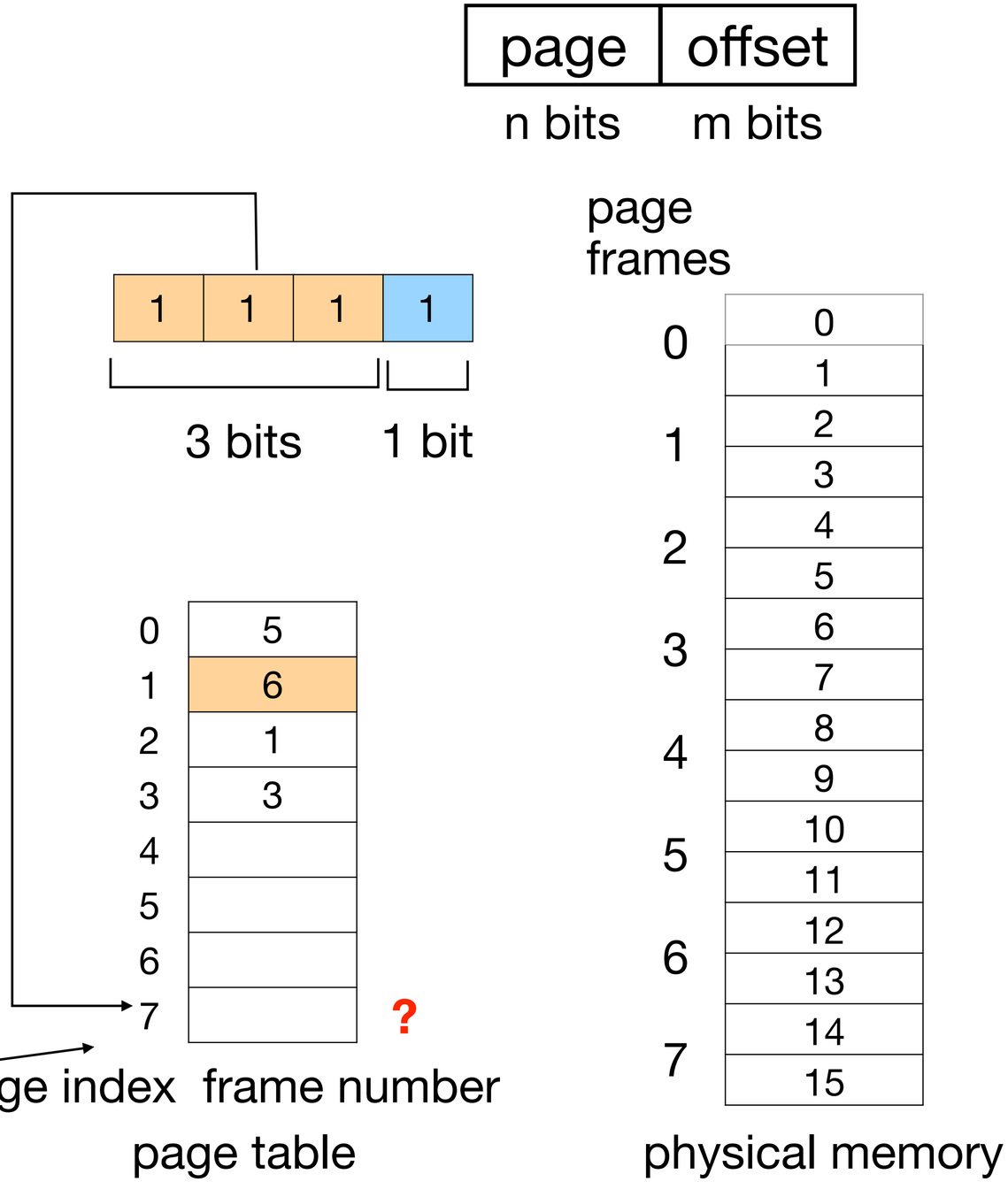
- 4-bit system - each address is 4 bits
- 2^4=16 different addresses
- 16 bytes of byte-addressable memory
- If we choose page size of 2 bytes
  - 16/2 = 8 pages
  - n = 3 - 3 bits to represent page
  - m = 1 - 1 bit for offset into the page



Paged addresses

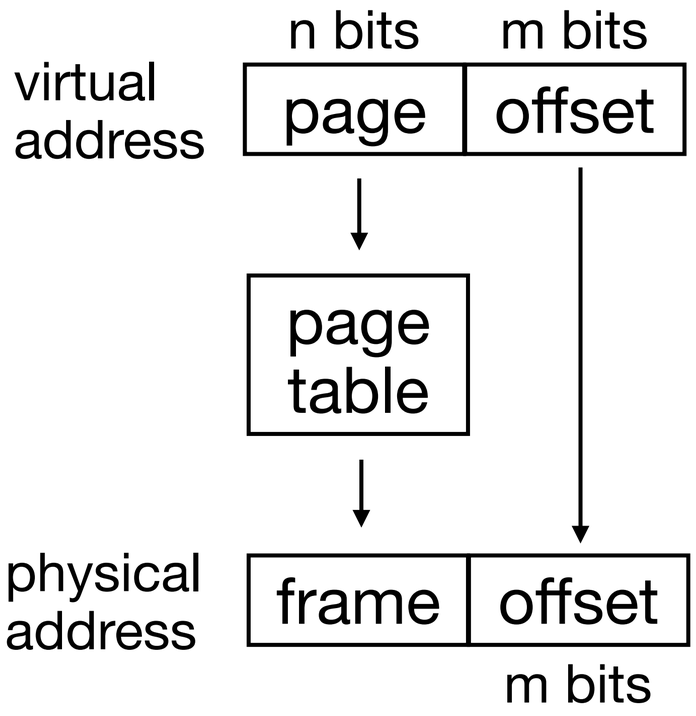
- 4-bit system - each address is 4 bits
- 2^4=16 different addresses
- 16 bytes of addressable memory
- If we choose page size of 2 bytes
  - 16/2 = 8 pages
  - n = 3 - 3 bits to represent page
  - m = 1 - 1 bit for offset into the page

page 7 is not mapped to physical memory - **page fault**



Paged addresses

- Computing physical address from virtual address
  - Extract first **n bits** to get a **virtual page number**
  - Look-up **physical page frame** in the **page table**
  - **Add** the remaining **m bits** of address to address of the page frame
  - Check if address is **out of bounds** (page size is known)



Page table organization

- Assume a linear page table - an array
- Array index is the virtual page number (VPN)

0	frame 3
1	...
...	...
x	frame Y

- Page-table entry (PTE) contains
  - physical frame number (PFN)
  - **valid bit** - marks invalid addresses in the virtual address space (e.g. free space between stack and heap)
  - **present bit** - if this page is in the physical memory
  - **dirty bit** - has this page been modified
  - **reference bit** (aka accessed bit) - has this page been accessed
  - **permissions** - is this address read-only or writable

Page tables in the wild

- Page tables in our examples are **tiny**
- Real page tables are **large**
  - e.g. 32-bit address space with 4KB-sized pages
    - will need 12 bits to address 4KB=4096=2^12 offsets
    - 20 bits for virtual pages = ~1M (2^20) translations in the table
    - each entry is 4 bytes -> 4MB memory
- Page tables are also **per process**
  - For 100 processes -> **400 MB** memory will be needed just for tables

Paging

- How many operations are needed to access memory with paging? MANY

```
VPN = (virtual_address & VPN_MASK) >> SHIFT

PTEAddr = PTBR + (VPN * sizeof(PTE))

PTE = AccessMemory(PTEAddr)

if (PTE.Valid == False)
    RaiseException(SEGMENTAION_FAULT)
else if (CanAccess(PTE.ProtectBits) == False)
    RaiseException(PROTECTION_FAULT)
else
    //access OK
    offset = virtual_address & OFFSET_MASK
    phys_addr = (PTE.PFN << PFN_SHIFT) | offset
    register = AccessMemory(phys_addr)
```

get the virtual page number
get address of the page table entry (PTE) - VPN is the offset

get the contents of the PTE

if this page is marked invalid - segfault

if this page is protected - abort

otherwise, get offset from the virtual address

combine it with the physical frame number (PFN)

and access physical memory address

Summary so far

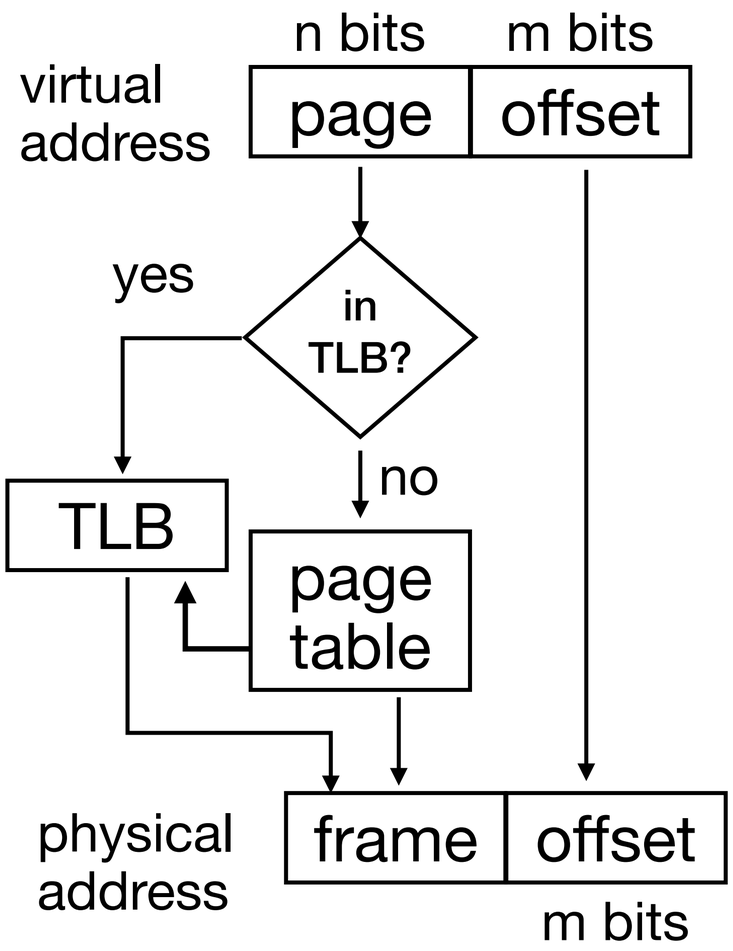
- Paging is similar to segmentation but with all chunks being **equal**
  - pros:
    - fixed-size pages avoid **external segmentation**
    - **easy to allocate** memory (all blocks are the same, any free one will do)
    - does **not** require hardware support - flexible
  - cons:
    - does not eliminate **internal fragmentation** (if page size is too big)
    - **page tables** are large and need a lot of memory to store
    - accessing memory is **slow** (**doubles** the number of memory operations per instruction)

Why bother with paging?

- Allows allocating more memory than physically available
  - **Swapping** - using the hard-drive as an extension to RAM
- Optimized data sharing - pages can be shared between multiple processes and threads
  - **copy-on-write** - when a process is fork'ed, copy of its address space is allocated only when one of the processes writes to it
- **Optimized memory access** - do not allocate memory for a given page until it is used/written to
  - Example #1 after you malloc'ed 2GB of memory, the OS will just add a placeholder to the page table and will allocate when you start writing to it
  - Example #2 **load-on-use** - load only the code that is about to be executed, do not load the rest

Paging: TLBs

- Problem: **too many memory accesses** needed for paging
- Solution: translation-lookaside buffer (**TLB**)
  - **cache** of virtual-to-physical address translations
  - check if the virtual address is in the TLB
    - if yes, then **avoid page table lookup** altogether
    - if no, perform page table lookup and cache the result



Paging: TLBs

```
VPN = (virtual_address & VPN_MASK) >> SHIFT
(success, TlbEntry) = TLB_Lookup(VPN)
if (success == True)
    if (CanAccess(TlbEntry.ProtectBits) == True)
      //access OK
      offset = virtual_address & OFFSET_MASK
      phys_addr = (PTE.PFN << PFN_SHIFT) | offset
      register = AccessMemory(phys_addr)
    else
      RaiseException(PROTECTION_FAULT)
else
    //compute physical address with page table
    //if valid address, insert into TLB
    TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
```

TLB hit

TLB miss

get the virtual page number

look up in the TLB

if found, check whether the address is OK to access

access memory

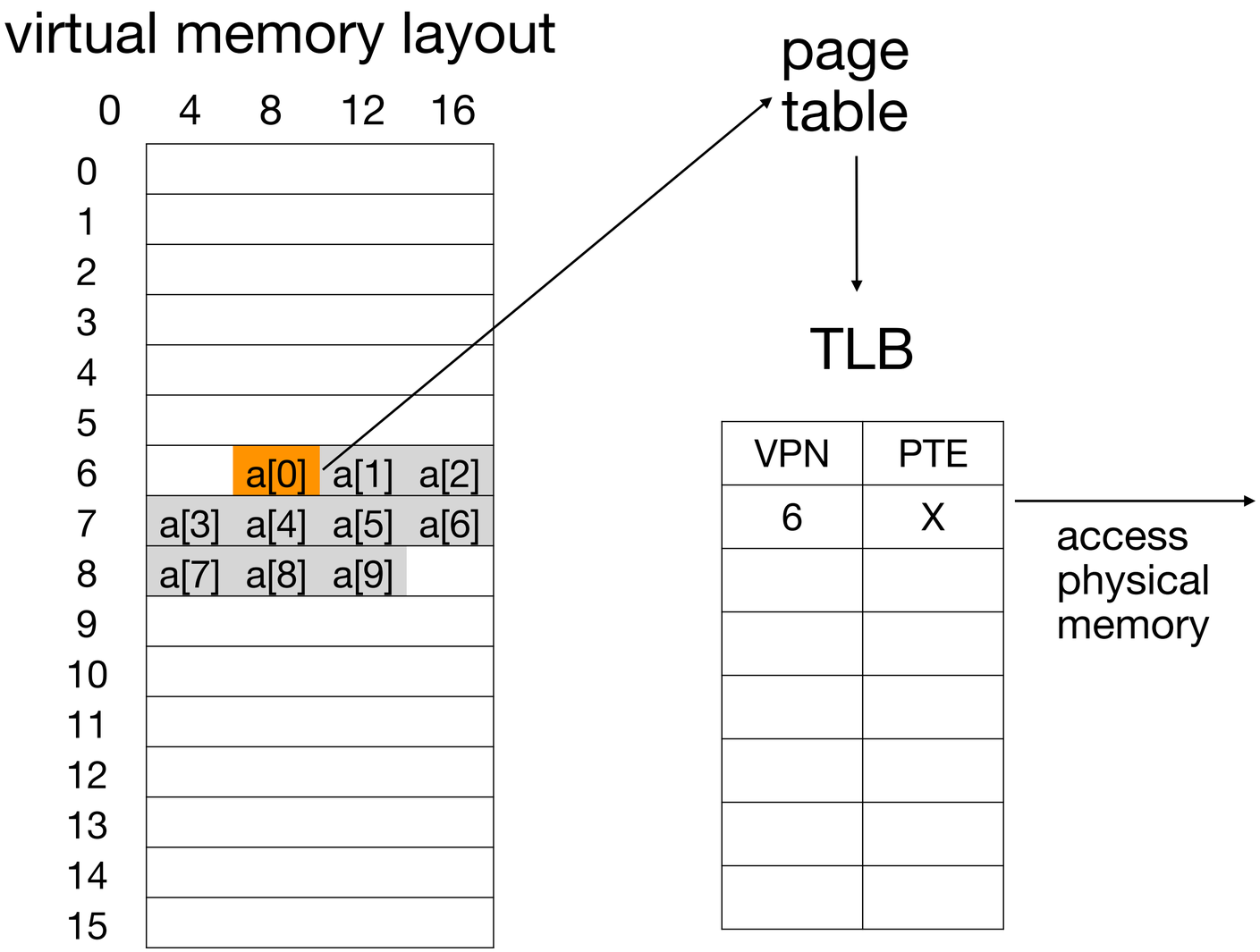
otherwise get page table entry (PTE) (code 3 slides ago)

if the address is valid, insert it into the TLB

Paging: TLBs

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

a[0] -> VPN=6, offset=4 -> TLB miss



16-byte pages, 8 bit addresses

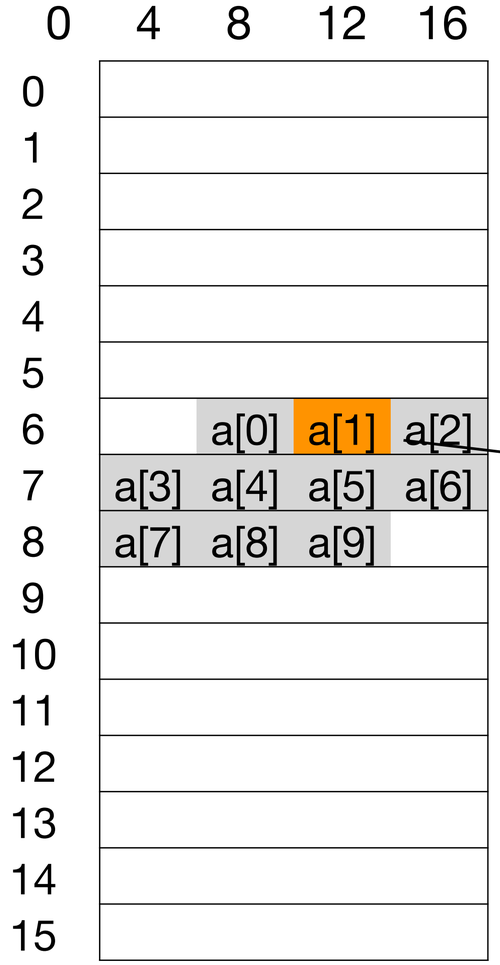
Paging: TLBs

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

a[0] -> VPN=6, offset=4 -> TLB miss

a[1] -> VPN=6, offset=8 -> TLB hit

virtual memory layout



page table

TLB

VPN	PTE
6	X

16-byte pages, 8 bit addresses

Paging: TLBs

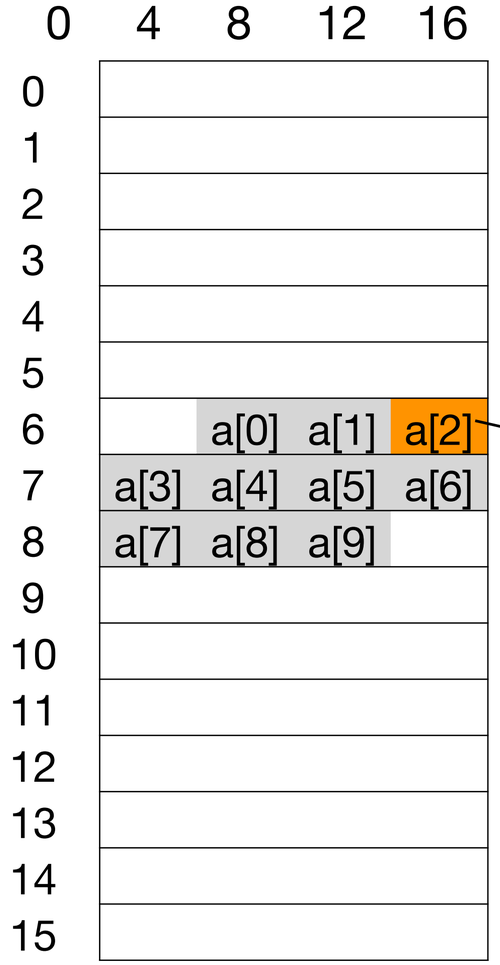
```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

a[0] -> VPN=6, offset=4 -> TLB miss

a[1] -> VPN=6, offset=8 -> TLB hit

a[2] -> VPN=6, offset=12 -> TLB hit

virtual memory layout



page table

TLB

VPN	PTE
6	X

16-byte pages, 8 bit addresses

Paging: TLBs

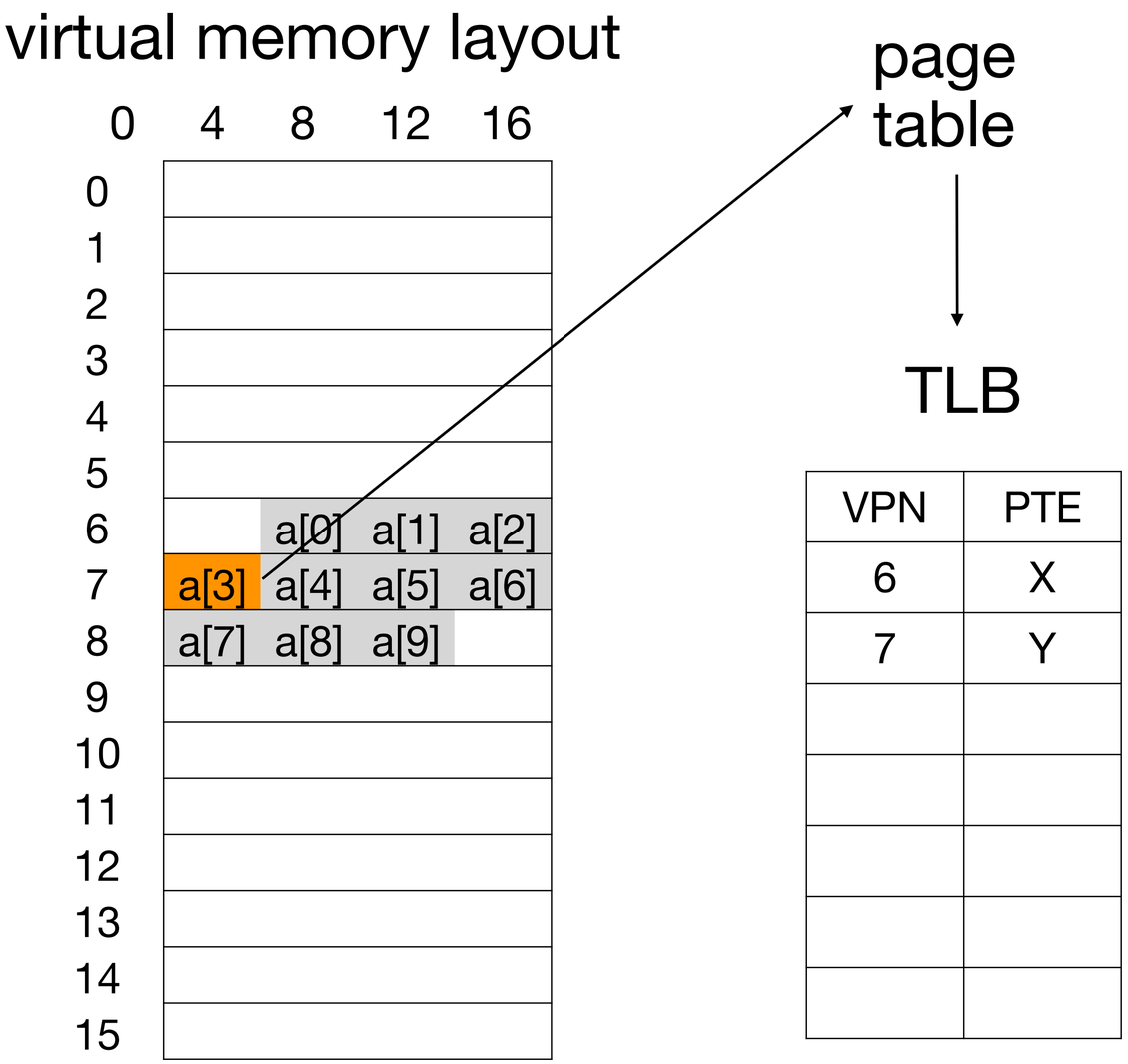
```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

a[0] -> VPN=6, offset=4 -> TLB miss

a[1] -> VPN=6, offset=8 -> TLB hit

a[2] -> VPN=6, offset=12 -> TLB hit

a[3] -> VPN=7, offset=0 -> TLB miss



16-byte pages, 8 bit addresses

Paging: TLBs

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

a[0] -> VPN=6, offset=4 -> TLB miss

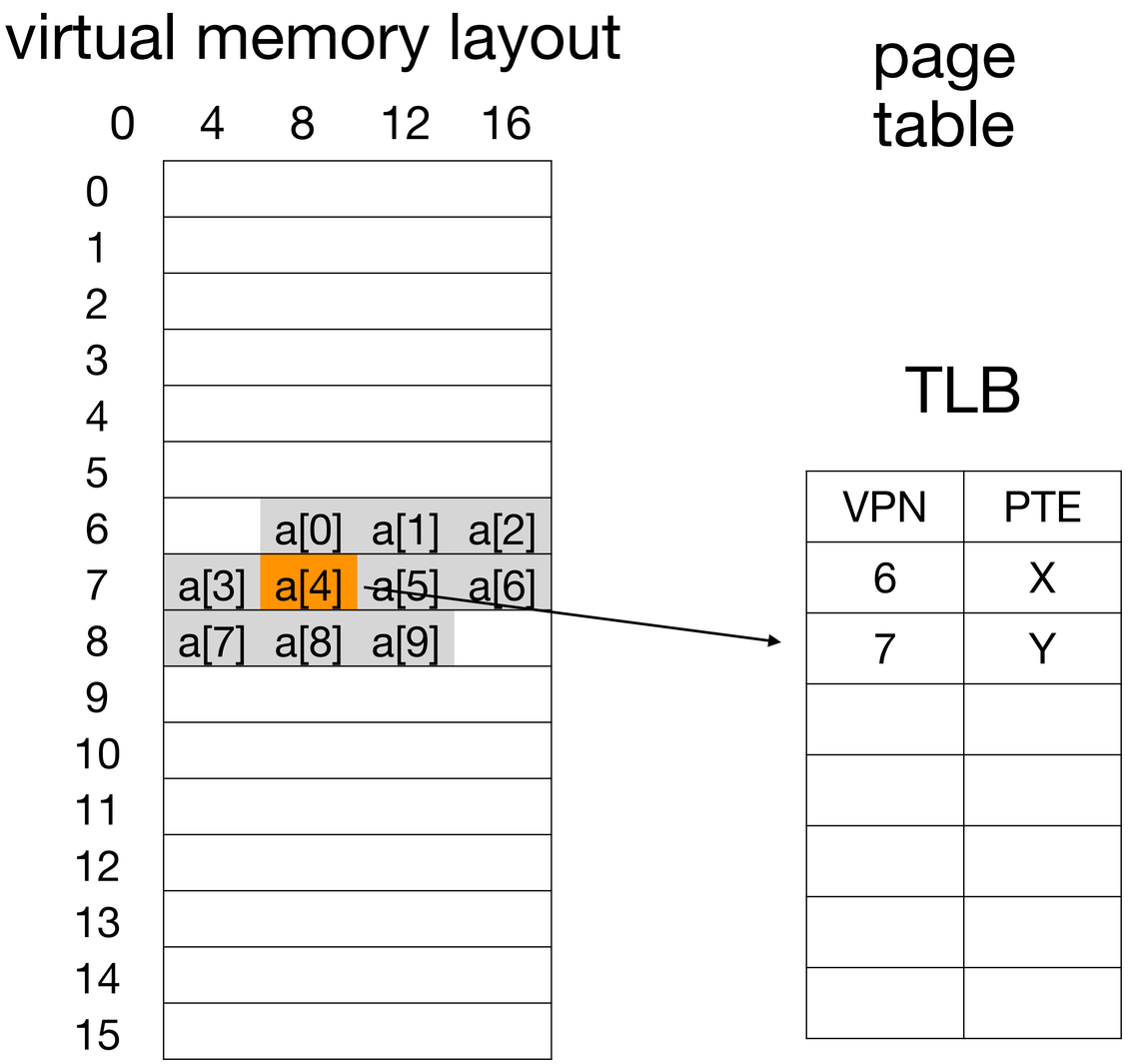
a[1] -> VPN=6, offset=8 -> TLB hit

a[2] -> VPN=6, offset=12 -> TLB hit

a[3] -> VPN=7, offset=0 -> TLB miss

a[4] -> VPN=7, offset=4 -> TLB hit

...



16-byte pages, 8 bit addresses

Paging: TLBs

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

a[0] -> VPN=6, offset=4 -> TLB miss

a[1] -> VPN=6, offset=8 -> TLB hit

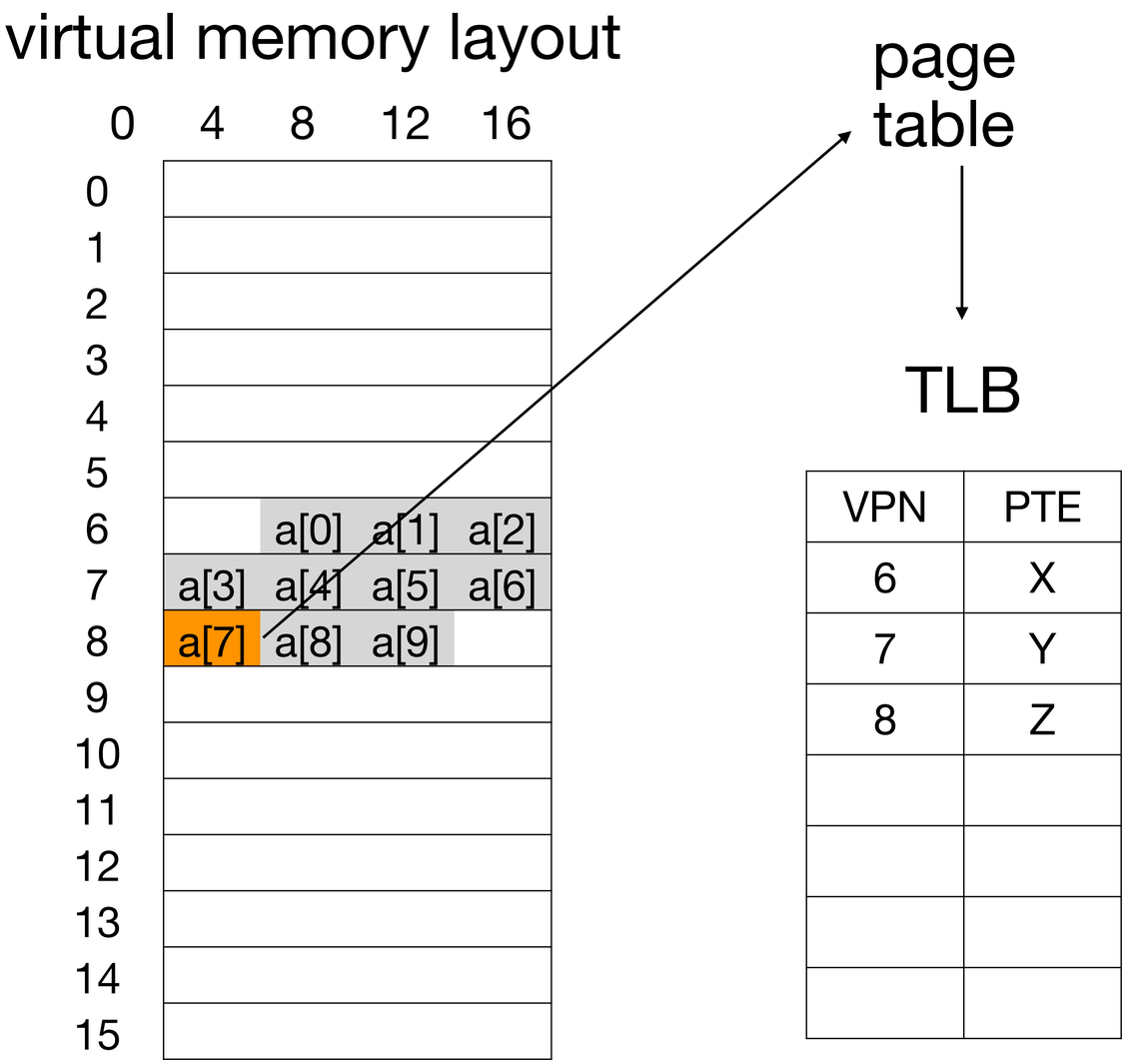
a[2] -> VPN=6, offset=12 -> TLB hit

a[3] -> VPN=7, offset=0 -> TLB miss

a[4] -> VPN=7, offset=4 -> TLB hit

...

a[7] -> VPN=8, offset=3 -> TLB miss



16-byte pages, 8 bit addresses

Paging: TLBs

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

- TLB hit rate - $\frac{\#\text{ of TLB hits}}{\#\text{ of memory accesses}}$
  - In the previous example
    - 10 accesses - for each array element
    - 3 misses, 7 hits
    - 70% TLB hit rate

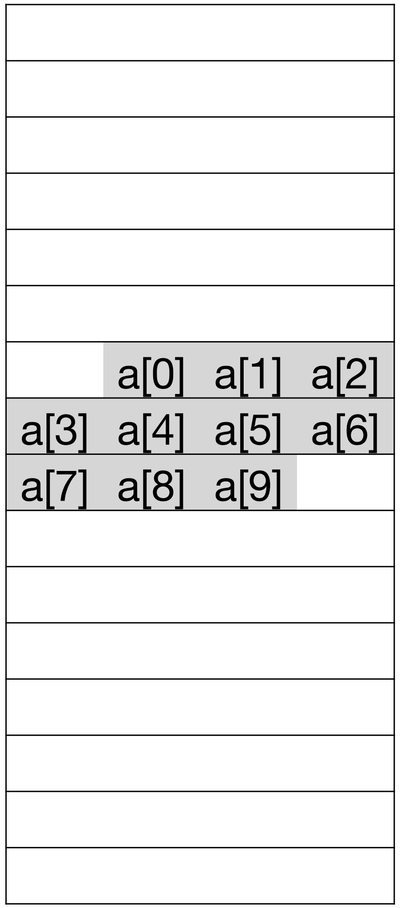
Paging: TLBs

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

- TLB hit rate - $\frac{\#\text{ of TLB hits}}{\#\text{ of memory accesses}}$
  - In the previous example
    - 10 accesses - for each array element
    - 3 misses, 7 hits
    - 70% TLB hit rate
- **Spatial locality** - accessing multiple values that are on the same page - close spatially
  - Larger page sizes help

elements are tightly packed together - close in space

only the first element in each page triggers a miss



Paging: TLBs

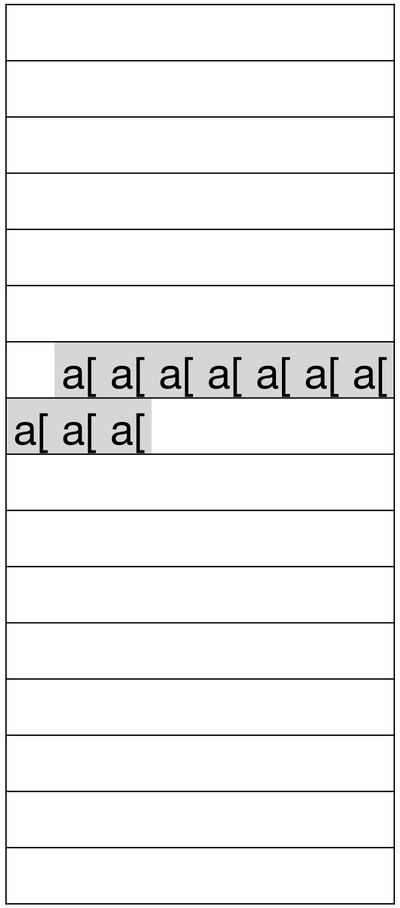
```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

- TLB hit rate - $\frac{\#\text{ of TLB hits}}{\#\text{ of memory accesses}}$
  - In the previous example
    - 10 accesses - for each array element
    - 3 misses, 7 hits
    - 70% TLB hit rate
- **Spatial locality** - accessing multiple values that are on the same page - close spatially
  - Larger page sizes help

elements are tightly packed together - close in space

only the first element in each page triggers a miss

packing more values into larger pages improves spatial locality



Paging: TLBs

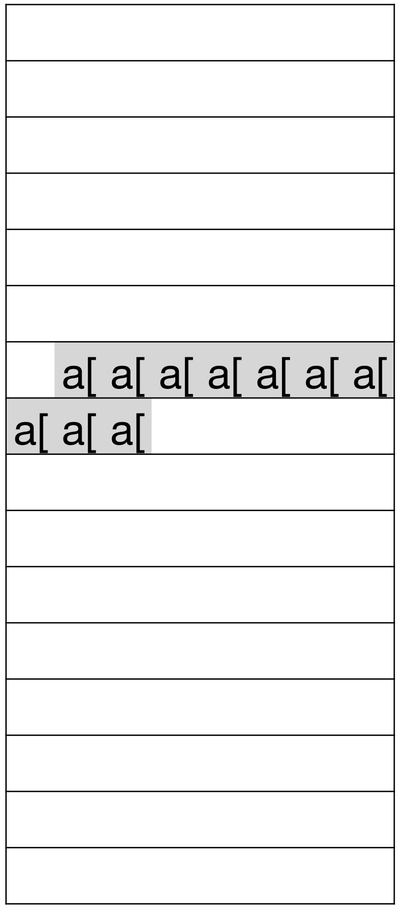
```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

- TLB hit rate - $\frac{\#\text{ of TLB hits}}{\#\text{ of memory accesses}}$
  - In the previous example
    - 10 accesses - for each array element
    - 3 misses, 7 hits
    - 70% TLB hit rate
- **Spatial locality** - accessing multiple values that are on the same page - close spatially
  - Larger page sizes help
- **Temporal locality** - accessing the same values multiple times within a **short time**
  - e.g. repeat the loop

elements are tightly packed together - close in space

only the first element in each page triggers a miss

packing more values into larger pages improves spatial locality



TLB miss

- How should TLB misses be handled?
  - Option #1: **hardware**
    - needs to know the location of the page tables in memory (page-table base register)
    - needs to know the format of the page table
    - would walk the page table, find the entry, extract address, and update the TLB

TLB miss

- How should TLB misses be handled?
  - Option #2: **software** (the OS)
    - raise an exception, go to kernel mode, and jump to a trap handler
    - lookup the translation in the page table, update the TLB and return from the trap
      - need to **redo the instruction that caused the trap**
    - retry the instruction (should be a TLB hit now)

TLB context switch

run one process

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

then start another

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

pid=1

a[0] -> VPN=6, offset=4 -> TLB miss

…

a[3] -> VPN=7, offset=0 -> TLB miss

TLB

VPN	PTE
6	X
7	Y

pid=2

a[0] -> VPN=6, offset=4 -> TLB hit?

virtual addresses may be the same, but physical addresses are different

How can we have TLB with context switching?

TLB context switch

- Problem: there is **one TLB but many processes**. How does TLB know which address to translate?
- Option #1: **flush** the TLB on context switch

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

pid=1

a[0] -> VPN=6, offset=4 -> TLB miss

a[3] -> VPN=7, offset=0 -> TLB miss

TLB

VPN	PTE
6	Z

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

pid=2

a[0] -> VPN=6, offset=4 -> TLB hit?

the second process gets an **empty TLB** any problems with this?

YES, many TLB misses after every context switch

TLB context switch

- Problem: there is **one TLB but many processes**. How does TLB know which address to translate?
- Option #2: **share** the TLB

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

pid=1

a[0] -> VPN=6, offset=4 -> TLB miss

a[3] -> VPN=7, offset=0 -> TLB miss

address space identifier for each TLB entry

TLB

VPN	PTE	**ASID**
6	X	1
7	Y	1
6	Z	2

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i]
}
```

pid=2

a[0] -> VPN=6, offset=4 -> TLB hit?

TLB context switch

- Problem: there is **one TLB but many processes**. How does TLB know which address to translate?
- Option #1: **flush** the TLB on context switch
  - pros: simple and effective - no program will every go to a wrong address
  - cons: this is **costly** if there are many context switches
    - every time the new program is run, it will have **many TLB misses**
- Option #2: **share** TLB across context switch
  - add a field to TLB for **address space identifier (ASID)**
  - use different ASID for TLB entries of different processes
  - pros: **fewer TLB misses** after context switching
  - cons: cache will **fill up** quicker

TLB replacement

- Problem: TLB cache **is small**. When TLB fills up, which entry should be **replaced**?
  - Option #1: **FIFO** - evict the oldest page
    - cons: more likely to throw out heavily used pages
  - Option #2: **MIN** - evict the page that will not be used in the future
    - cons: need to predict the future
  - Option #3: **LRU** - evict the page that has not been used the longest
    - a good approximation of MIN
  - Option #4: **Random** - evict a page chosen at random
    - a lower bound on the TLB replacement policy

TLB contents

- Typical TLB size is 12 bits - **4096 entries**
- **Fully-associative** - translation can be anywhere in the TLB - hardware will search all of it
- TLB location - part of **MMU**, sits between CPU and RAM, can be more than one TLB
- TLB entry contains
  - **VPN** - virtual page number
  - **PFN** - physical frame number
  - **protection** bits - write, read, execute permissions for the page
  - address-space identifier (**ASID**) - for different processes
  - **dirty bit** - has the page been modified
  - **reference bit** - has the page been accessed (useful for replacement)
  - and more

Summary

- **Paging** splits memory into **equally-sized** chunks — simpler and more efficient memory allocation
  - Pros
    - fixed-size pages avoid **external segmentation**
    - **easy to allocate** memory (all blocks are the same, any free one will do)
    - does **not** require hardware support - flexible
  - Cons
    - does not eliminate **internal fragmentation** (if page size is too big)
    - page tables are **large**
    - accessing memory is **slow** (**doubles** the number of memory operations per instruction) -> TLB helps with this

Summary

- **Translation Lookaside Buffer (TLB)** - memory **cache** for storing translations of virtual memory to physical memory addresses
  - Pros
    - **reduces** memory accesses to page table
    - **very fast** hardware-supported implementation
  - Cons
    - needs **careful management** to avoid too many **TLB misses**

Reading

- Chapter 18 Paging: Introduction
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-paging.pdf
- Chapter 19 Paging: Faster Translations (TLBs)
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-tlbs.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/vm-paging
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/vm-segmentation
- No optional reading this week

Thank you

- To those of you who showed up for the URA/USRA Research Mixer!
- I will have other research opportunities for S26
  - More info closer to the end of the term

A1

- Was due on **Feb 1 @ 11:59 PM EST**
- 238/260 students submitted so far
- **80%** of the grade will be based on the autograder results
- Submissions without the autograder report will receive 0
  - If you did not get the autograder to work, we are not going to be rerunning the tests
- Manual grading will start after the grace day period ends
  - Speed test #71 results by Feb 10 lecture
  - README feedback before Feb 15

A2 is posted

- Due on **Feb 15 @ 11:59 PM EST**
- Start yesterday (5 students already started)
- Few notes for development:
  - you do not need a remote server
  - just clone the gitlab repo, develop locally, and push to get feedback
  - push changes frequently - will help identify and fix problems early
  - [very optional] set up a docker to test in a Linux environment

A2 is posted

- Deliverables
  - **[10 points]** - bump allocator
  - **[50 points]** - free-list allocator with first-fit, best-fit, and worst-fit strategies
  - **[10 points]** - shorter free-list allocator than my implementation
    - `cloc` will be used - counts **only** lines of code (you can leave comments and empty lines, they won't be counted)
    - my implementation is not optimized for LOC at all, will be easy to beat
    - do not try to game the LOC metric, work on writing more compact code
  - **[0-10 points]** - the shortest free-list allocator will get 10, the rests proportional
  - **[10 points]** - README.md

README

- Why do I care? Why should you care?
- README is a business card for your work - whether you want it or not, it will reflect on you
- A good README will **increase the chances of your project being used**
- A good README will **help you reuse code later**
- Do not write README for me, **write it for anyone** who can potentially come across this project
  - Assume prior knowledge of programming
  - Make the information **useful, easy to find, and easy to understand** — good formatting, good grammar
  - Make it **neat and attractive**

Quiz 7

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999